Объектно-ориентированное программирование

6 семестр
Лекция 4.

Содержание лекции:

- Поведенческие паттерны
- Стратегия (Strategy)
- Наблюдатель (Observer)
- Команда (Command)
- Состояние (State)

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Поведенческие паттерны

Если порождающие паттерны отвечают за «рождение» объектов, а структурные — за их «скелет» (связи), то **поведенческие паттерны** определяют их **характер, логику действий и способы общения.**

Суть группы:

Эти шаблоны проектирования направлены на то, чтобы сделать взаимодействие между объектами максимально гибким и эффективным.

Что они решают:

- **Распределение обязанностей:** Кто именно в системе должен выполнять конкретную задачу?
- **Алгоритмы взаимодействия:** Как объекты должны реагировать на действия друг друга, не становясь при этом жестко зависимыми?
- **Управление потоком данных:** Как эффективно передавать информацию между разными частями программы.

Стратегия (Strategy)

Проблема: «Раздутые» классы и гигантские if-else

Представьте класс Car. Если мы пропишем логику движения (бензин, электро, газ) прямо внутри него через switch, то при добавлении нового типа двигателя нам придется переписывать весь класс.

Решение: Вынос алгоритма в отдельный объект

Паттерн Стратегия определяет семейство схожих алгоритмов, инкапсулирует каждый из них и делает их взаимозаменяемыми.

Суть «убийцы if-else»:

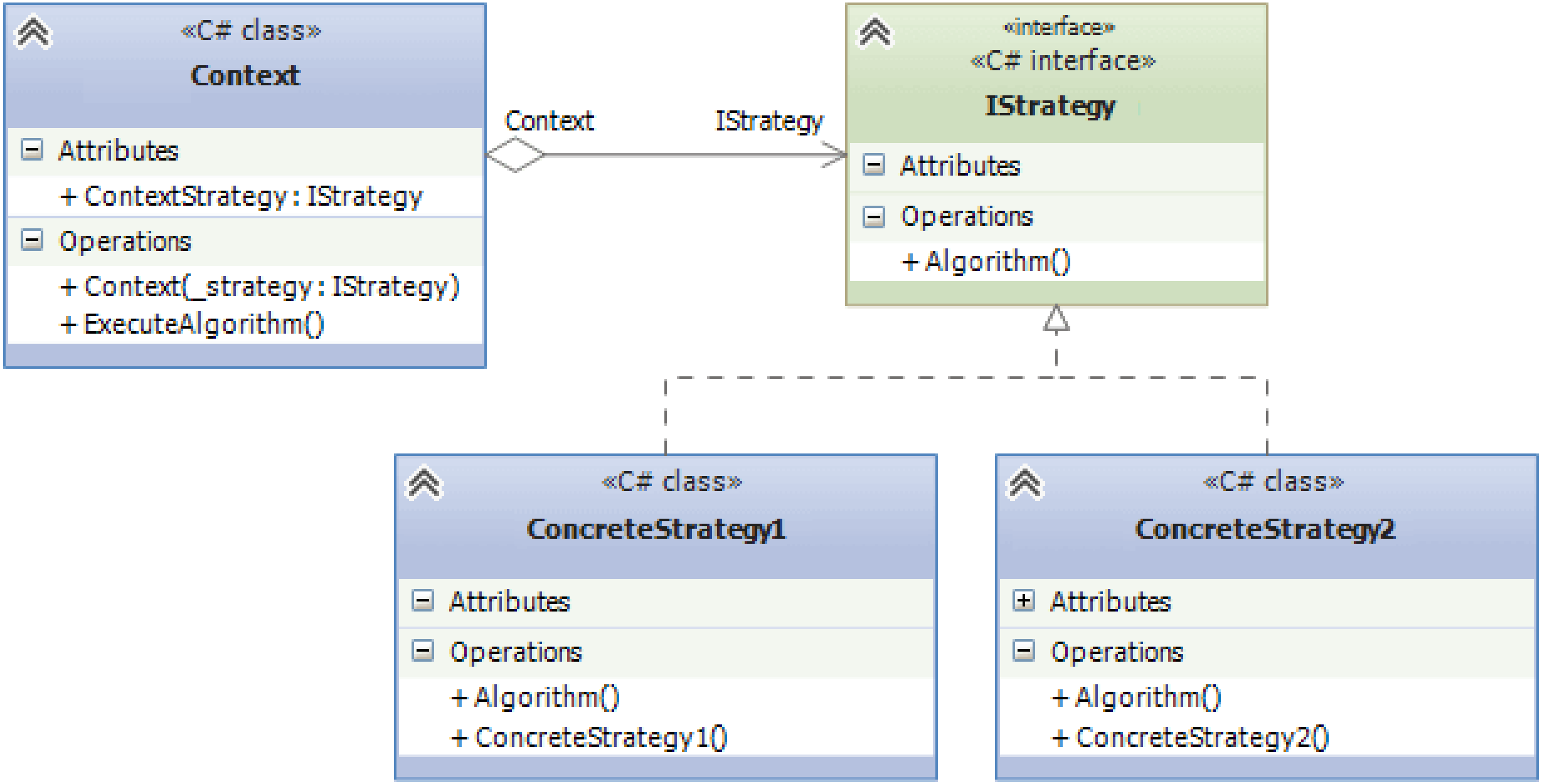
Вместо того чтобы спрашивать объект «Как ты двигаешься?», мы просто даем ему нужный «инструмент» (стратегию) для движения.

Интерфейс Стратегии (IStrategy): Общий интерфейс для всех вариаций алгоритма.

Конкретные стратегии: Реальные реализации алгоритма (например, BubbleSort или QuickSort).

Контекст (Context): Класс, который использует стратегию. Он хранит ссылку на интерфейс и не знает, какая именно реализация подсунута ему в данный момент.

Главный плюс: Мы можем изменить поведение объекта прямо «на лету», просто подменив одну стратегию на другую.


«C# class»
Context
Attributes
+ ContextStrategy : IStrategy
Operations
+ Context(_strategy : IStrategy)
+ ExecuteAlgorithm()
Context
IStrategy
«interface»
«C# interface»
IStrategy
Attributes
Operations
+ Algorithm()
«C# class»
ConcreteStrategy1
Attributes
Operations
+ Algorithm()
+ ConcreteStrategy1()
«C# class»
ConcreteStrategy2
Attributes
Operations
+ Algorithm()
+ ConcreteStrategy2()

```
// 1. Интерфейс - общий для всех типов движения
interface IMovable { void Move(); }
// 2. Конкретные способы реализации
class PetrolMove : IMovable
{
    public void Move() => Console.WriteLine("Едем на бензине");
}
class ElectricMove : IMovable
{
    public void Move() => Console.WriteLine("Едем на
электричестве");
}
// 3. Контекст - машина, которой всё равно, какой внутри двигатель
class Car
{
    public IMovable Movable { get; set; } // Сменная стратегия
    public void Move() => Movable.Move();
}
```

```
// 1. Создаем машину с бензиновым двигателем
Car myCar = new Car
{
    Movable = new PetrolMove() // Устанавливаем начальную
стратегию
};
myCar.Move(); // Выведет: "Перемещение на бензине"

// 2. Смена стратегии ПРЯМО В ПРОЦЕССЕ (например, гибрид
переключился)
myCar.Movable = new ElectricMove();
myCar.Move(); // Выведет: "Перемещение на электричестве"
```

```
// Единый стандарт для всех сортировок
public interface ISortStrategy
{
    void Sort(int[] array);
}

// Конкретные реализации (алгоритмы скрыты)
public class BubbleSort : ISortStrategy
{
    public void Sort(int[] array) { /* Логика пузырька */ }
}

public class QuickSort : ISortStrategy
{
    public void Sort(int[] array) { /* Логика быстрой
сортировки */ }
}
```

```
public class SortContext
{
    private ISortStrategy _strategy;

    public SortContext(ISortStrategy strategy) => _strategy =
strategy;

    // Позволяет сменить алгоритм в любой момент
    public void SetStrategy(ISortStrategy strategy) =>
_strategy = strategy;

    public void ExecuteSort(int[] array) =>
_strategy.Sort(array);
}
```

Паттерн часто применяется там, где выбор алгоритма зависит от условий (например, объема данных).

Мало данных? Используем BubbleSort (просто, мало памяти).

Много данных? Переключаемся на QuickSort (быстро).

```
int[] data = { 5, 1, 10, 2 };
SortContext sorter = new SortContext(new QuickSort());

sorter.ExecuteSort(data); // Используем быстрый алгоритм

sorter.SetStrategy(new BubbleSort());
sorter.ExecuteSort(data); // Тот же объект теперь сортирует
пузырьком
```

Преимущества (+):

- **Соблюдение Open/Closed Principle:** Можно добавлять новые алгоритмы, не меняя существующие классы.
- **Чистота:** Мы избавляемся от сложной условной логики (if-else / switch).
- **Изоляция:** Алгоритмы отделены от данных объекта, что упрощает их тестирование.

Недостатки (-):

- **Усложнение:** Увеличивается количество классов в проекте.
- **Осведомленность:** Клиент (программист) должен знать, чем отличаются стратегии, чтобы выбрать подходящую.

Наблюдатель (Observer)

Проблема: Ожидание изменений Как один объект может узнать, что в другом объекте произошло важное событие, не спрашивая об этом каждую секунду?

Постоянные опросы («Ну что, изменилось?») тратят ресурсы процессора и делают код запутанным.

Суть паттерна: Создается механизм «подписки», который позволяет объекту (**Издателю**) оповещать множество других объектов (**Подписчиков**) об изменениях своего состояния.

Аналогия из жизни:

Маркетплейсы: Вы добавляете товар в «Избранное». Когда цена на него падает, сервер рассылает уведомление всем заинтересованным покупателям.

Паттерн строится на двух ролях:

Издатель (Subject / Publisher):

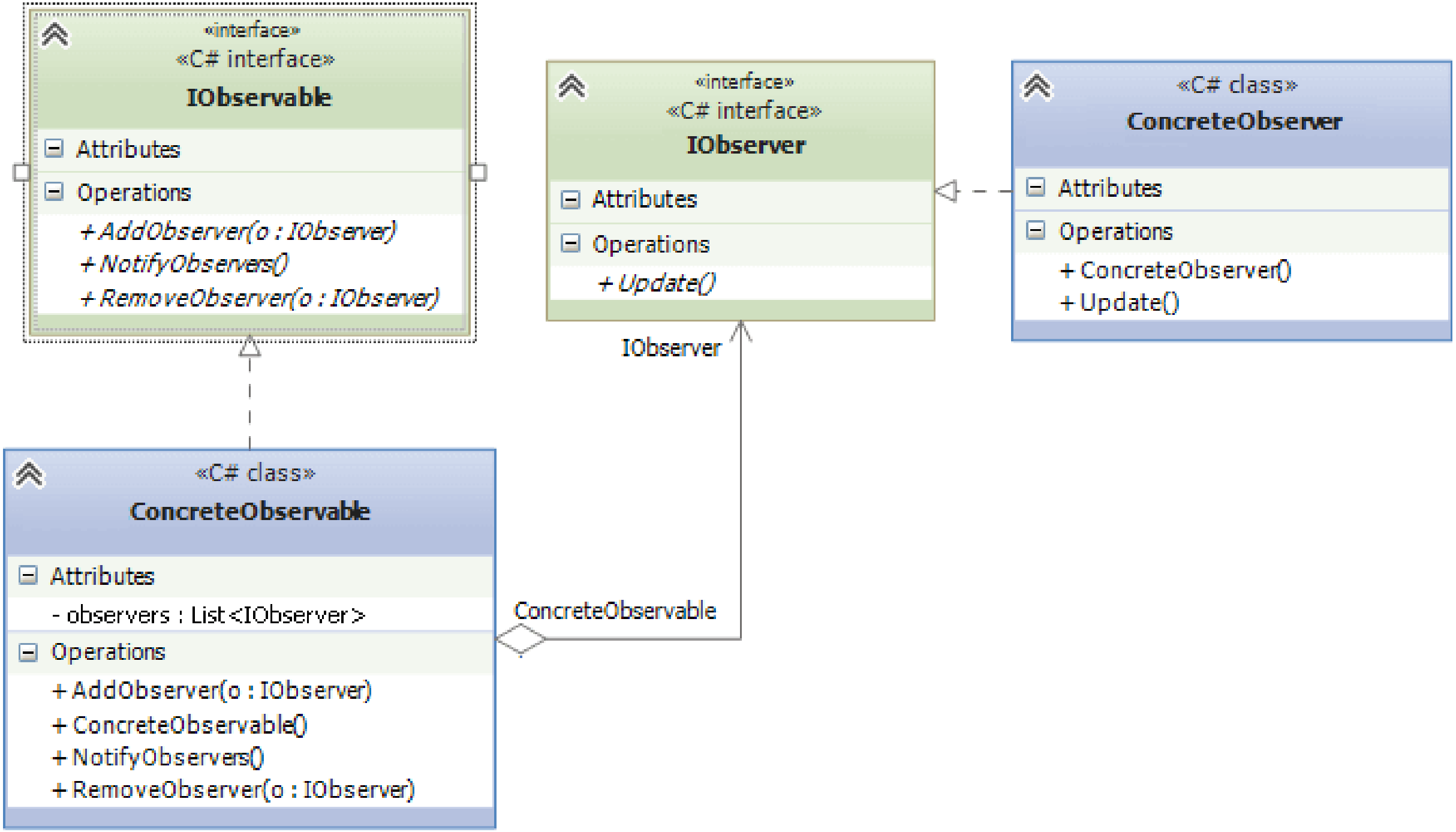
Хранит внутри себя **список** всех подписавшихся объектов.

Предоставляет методы для подписки (Attach) и отписки (Detach).

Имеет метод Notify, который проходит циклом по списку и вызывает у каждого подписчика метод обновления.

Подписчик (Observer / Subscriber):

Реализует общий интерфейс с методом (например, Update), который вызывается издателем.


«interface»
«C# interface»
IObservable
Attributes
Operations
+ AddObserver(o : IObserver)
+ NotifyObservers()
+ RemoveObserver(o : IObserver)
«interface»
«C# interface»
IObserver
Attributes
Operations
+ Update()
«C# class»
ConcreteObserver
Attributes
Operations
+ ConcreteObserver()
+ Update()
IObserver
«C# class»
ConcreteObservable
Attributes
- observers : List<IObserver>
Operations
+ AddObserver(o : IObserver)
+ ConcreteObservable()
+ NotifyObservers()
+ RemoveObserver(o : IObserver)
ConcreteObservable

Где вы это уже видели?

В Лекции 2 мы разбирали интерфейс **INotifyPropertyChanged**. Это и есть классическая реализация Наблюдателя в .NET!

ViewModel здесь выступает в роли **Издателя**.

Интерфейс (View) выступает в роли **Подписчика**.

Когда во ViewModel меняется свойство, она «выстреливает» событием, а UI-элементы, которые «слушают» это событие, мгновенно перерисовывают себя. На этом механизме держится вся современная разработка интерфейсов (WPF, Avalonia, Android, React).

```
// Интерфейс подписчика
interface IObserver
{
    void Update(string message);
}

class Customer : IObserver
{
    public string Name { get; }
    public Customer(string name) => Name = name;

    // Метод, который вызовет Издатель
    public void Update(string message)
    {
        Console.WriteLine($"Уведомление для {Name}:
{message}");
    }
}
```

```
// Издатель (Магазин)
class ProductStore
{
    private List<IObserver> _observers = new();

    public void Subscribe(IObserver s) => _observers.Add(s);

    public void Notify(string news)
    {
        foreach (var observer in _observers)
            observer.Update(news); // Рассылка всем!
    }

    public void ChangePrice() => Notify("Цена на iPhone
снижена!");
}
```

```
// Создаем Издателя (Магазин)
ProductStore appleStore = new ProductStore();

// Создаем Подписчиков (Клиентов)
Customer ivan = new Customer("Иван");
Customer anna = new Customer("Анна");

// Подписываем их на обновления
appleStore.Subscribe(ivan);
appleStore.Subscribe(anna);

// Произошло событие - магазин вызывает Notify()
appleStore.ChangePrice();
```

```
Уведомление для Иван: Цена на iPhone снижена!
Уведомление для Анна: Цена на iPhone снижена!
```

Преимущества (+):

- **Слабая связанность (Loose Coupling):** Издатель не знает, кто на него подписан и что они будут делать с информацией.
- **Динамика:** Можно добавлять и удалять подписчиков прямо во время работы программы.
- **Broadcast:** Позволяет реализовать массовую рассылку сообщений одним вызовом.

Недостатки (-):

- **Порядок уведомлений:** Подписчики получают уведомления в случайном порядке.
- **Утечки памяти:** Если забыть «отписаться» от издателя, объект-подписчик никогда не будет удален сборщиком мусора (это называют «Lapsed Listener problem»).

Команда (Command)

Проблема: Жесткая связь отправителя и получателя

Обычно, когда мы жмем на кнопку, она напрямую вызывает метод в коде. Это создает проблему:

Кнопка «знает» слишком много о логике приложения.

Трудно переиспользовать логику (например, одно и то же действие для кнопки и горячей клавиши).

Невозможно отменить действие.

Решение: Превратить действие в объект

Паттерн Команда инкапсулирует запрос в виде отдельного объекта.

Теперь «запрос на сохранение» — это не просто вызов метода, а самостоятельная сущность, которую можно передавать, хранить и ставить в очередь.

Аналогия: Заказ в ресторане

Вы (Отправитель) не идете на кухню объяснять повару (Получателю), как жарить мясо. Вы пишете **заказ на листке (Команда)**. Официанту всё равно, что в заказе, он просто относит его на кухню. Листок можно передать другому повару или аннулировать.

Как это работает? (Роли в паттерне)

Команда (Command): Интерфейс с методом Execute().

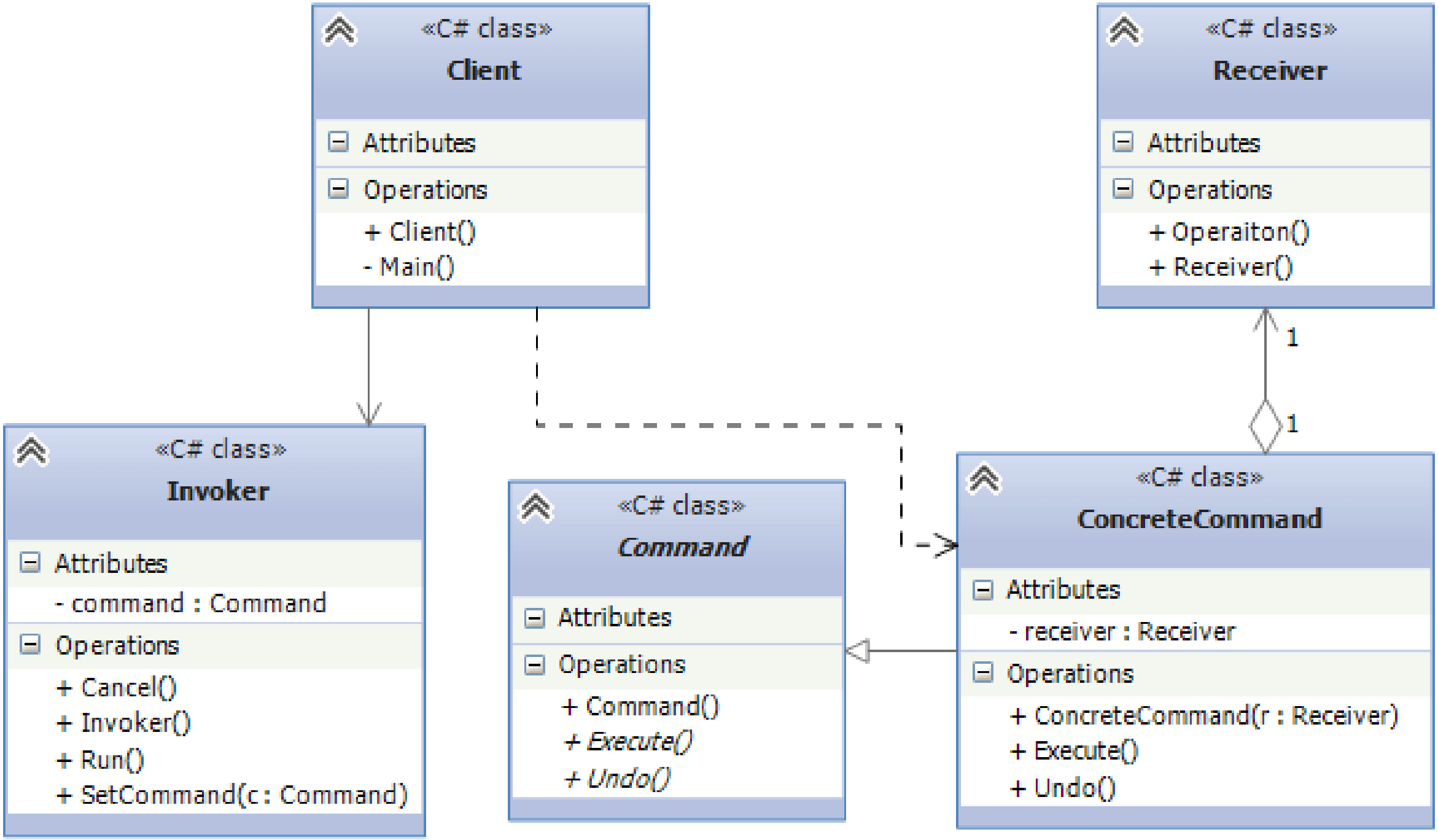
Конкретная команда: Реализует Execute, внутри которого вызывает метод у Получателя.

Получатель (Receiver): Класс, который реально знает, как выполнять работу (бизнес-логика).

Инициатор (Invoker): Объект, который просит команду выполниться (например, кнопка).

Киллер-фича: Отмена действия (Undo)

Если в интерфейс команды добавить метод Undo(), мы сможем сохранять выполненные команды в **Стек (Stack)**. Чтобы отменить последние 10 действий, нам нужно просто достать 10 команд из стека и вызвать у них Undo().


«C# class»
Client
Attributes
Operations
+ Client()
- Main()
«C# class»
Receiver
Attributes
Operations
+ Operaiton()
+ Receiver()
1
1
«C# class»
Invoker
Attributes
- command : Command
Operations
+ Cancel()
+ Invoker()
+ Run()
+ SetCommand(c : Command)
«C# class»
Command
Attributes
Operations
+ Command()
+ Execute()
+ Undo()
«C# class»
ConcreteCommand
Attributes
- receiver : Receiver
Operations
+ ConcreteCommand(r : Receiver)
+ Execute()
+ Undo()

```
// 1. Интерфейс с возможностью отмены
interface ICommand
{
    void Execute();
    void Undo();
}

// 2. Получатель (Логика телевизора)
class TV
{
    public void On() => Console.WriteLine("ТВ включен");
    public void Off() => Console.WriteLine("ТВ выключен");
}
```

```
// 3. Конкретная команда
class ToggleTVCommand : ICommand
{
    private TV _tv;
    public ToggleTVCommand(TV tv) => _tv = tv;

    public void Execute() => _tv.On();
    public void Undo() => _tv.Off();
}
```

```
TV myTv = new TV();
ICommand command = new ToggleTVCommand(myTv);

// Хранилище истории для Undo
Stack<ICommand> history = new Stack<ICommand>();

// Выполняем и запоминаем
command.Execute();
history.Push(command);

// Нажали Ctrl+Z (Отмена)
if (history.Count > 0)
{
    history.Pop().Undo(); // ТВ выключится сам!
}
```

Преимущества (+):

- **Слабая связанность:** Класс, вызывающий команду, и класс, выполняющий её, полностью разделены.
- **История действий:** Легко реализовать Undo/Redo и очередь команд.
- **Комбинирование:** Можно собирать сложные команды из нескольких простых (Макрокоманды).

Недостатки (-):

- **Много классов:** Для каждого отдельного действия приходится создавать свой класс команды. Это может «раздуть» проект.

Состояние (State)

Проблема: Зависимость поведения от статуса

У многих объектов есть внутреннее состояние, которое диктует, как они должны реагировать на команды.

Пример: Заказ в интернет-магазине. Метод должен работать, если заказ «**Новый**», и выдавать ошибку, если он уже «**В пути**» или «**Доставлен**».

Как это обычно пишут (плохо):

Бесконечные проверки внутри каждого метода:

```
public void Cancel()
{
    if (status == "New") { /* отменить */ }
    else if (status == "Shipped") { /* ошибка */ }
    else if (status == "Canceled") { /* уже отменен */ }
    // ... и так в каждом методе!
}
```

Паттерн **Состояние** позволяет объекту менять поведение при изменении его внутреннего состояния. Кажется, будто объект сменил свой класс.

Реализация:

- **Интерфейс Состояния (IState):** Описывает все возможные действия (например, PushPowerButton, InsertCard).
- **Классы состояний:** Каждое состояние (например, PowerOnState, WaitingForPinState) реализует этот интерфейс **по-своему**.
- **Контекст (Сам объект):** Хранит ссылку на текущий объект-состояние и просто перенаправляет ему все вызовы.

Аналогия: Смартфон на зарядке

Поведение кнопки питания зависит от уровня заряда и статуса:

Состояние «Разряжен»: Нажатие кнопки — на экране иконка пустой батареи.

Состояние «Заряжается»: Нажатие кнопки — экран показывает процент заряда.

Состояние «Включен»: Нажатие кнопки — экран гаснет или загорается.

Кнопка одна, а реализация метода PressPowerButton() в каждом классе-состоянии своя. Смартфону не нужно проверять if, он просто вызывает метод у текущего состояния.

```
// Интерфейс состояния
interface IOrderState
{
    void Cancel(Order order);
}

// Конкретное состояние: Новый заказ
class NewOrderState : IOrderState
{
    public void Cancel(Order order)
    {
        Console.WriteLine("Заказ отменен.");
        order.SetState(new CanceledState());
    }
}
```

```
// Конкретное состояние: Отменен
class CanceledState : IOrderState
{
    public void Cancel(Order order)
    {
        // Логика: нельзя отменить то, что уже отменено
        Console.WriteLine("Ошибка: Заказ уже был отменен
ранее.");
    }
}

// Конкретное состояние: Доставлен
class DeliveredOrderState : IOrderState
{
    public void Cancel(Order order) =>
        Console.WriteLine("Ошибка: нельзя отменить доставленный
заказ!");
}
```

```
class Order
{
    private IOrderState _state;
    public Order(IOrderState initialState) => _state =
initialState;

    public void SetState(IOrderState state) => _state = state;

    // Просто делегируем работу текущему состоянию
    public void Cancel() => _state.Cancel(this);
}

// В программе:
Order myOrder = new Order(new NewOrderState());
myOrder.Cancel(); // "Заказ отменен"
myOrder.Cancel(); // "Ошибка: нельзя отменить доставленный
заказ" (уже другое состояние)
```